

소프트웨어 프로젝트 최종보고서

팀 명	UCI		
작 품 명	mU-Code Interpreter		
개발기간	2025년 10월 27일 ~ 2025년 12월 02일		
지도교수	컴퓨터공학전공		윤OO
구 분	학년	학 번	성 명
책임자(팀장)	2	2022----	이강석
팀원	2	2024----	권기영

소프트웨어프로젝트 과목에서 수행한 프로젝트 최종보고서를 첨부와 같이 제출합니다.

2025 년 12 월 15 일

국립금오공과대학교 컴퓨터공학부 컴퓨터공학전공

프로젝트 최종보고서

mU-Code Interpreter

목차

1. 서론	4
2. 요구사항 및 개발환경	6
2.1 기능 요구사항	6
2.2 비기능 요구사항	7
3. 설 계	9
3.1 프로그램 구조 설계	9
3.1.1 모듈 설계	9
3.1.2 모듈간 관계	10
3.2 UI 설계	10
3.2.1 CLI	10
3.2.2 GUI	11
3.3 자료 구조 설계	11
3.3.1 자료 구조 설계 개요	11
3.3.2 가상 머신 구조	12
3.3.3 세부 자료 구조	12
3.4 알고리즘 설계	13
3.4.1 알고리즘 설계 개요	13
3.4.2 주요 알고리즘 flowchart/sequence diagram	14
4. 구 현	17
4.1 자료 구조 구현	17
4.2 핵심 소스 코드	20
4.2.1 어셈블 코드	20
4.2.2 실행 엔진 코드	22
4.2.3 GUI 스레드 및 입출력 비동기 코드	23
4.3 실행 화면	24

4.4 결과 분석	27
4.4.1 실험 조건	27
4.4.2 버블 정렬	28
4.4.3 이진 트리 DFS	28
4.4.4 피보나치 수열	29
4.4.5 메모리 사용량 비교	29
4.4.6 분석	30
5. 결 론	31
5.1 핵심 성과 요약	31
5.2 의의 및 활용 가능성	31
5.3 한계점 및 향후 개선 과제	31
5.4 기대 효과	31
6. 자체 평가서	32
6.1 자체 분석 및 평가	32
6.2 팀원 역할 및 기여도	33
7. 부 록	34
8. 참고문헌	35

1. 서론

본 프로젝트는 Pascal 언어의 중간 코드로 설계된 P-Code를 일반화하고 간략화한 mU-Code를 실행할 수 있는 인터프리터와 가상 머신 구조를 설계 및 구현하는 것이다. 기본적으로 mU-Code를 해석해 실행하는 인터프리터를 구현하고, 파일 기반의 mU-Code 입력, mU-Code의 실행, 실행 결과 및 통계, 단계별 실행 과정, 실행 단계에서의 오류 감지 기능 그리고 GUI 환경 등을 모두 제공하는 프로그램을 구현하는 것을 이 프로젝트의 목표로 한다.

중간 언어는 고급 언어에서 기계어로 변환하는 컴파일 과정의 중간 단계로, 다양한 고급 언어와 다양한 기계어 사이를 연결하는 역할을 한다. 서로 다른 언어로 작성된 프로그램이라도 같은 중간 언어로 바꾸는 과정을 이용하면 여러 CPU에서 명령어를 처리하는 방법이 다르더라도 같은 중간 언어를 사용하기 때문에 문제 없이 실행할 수 있다. 즉, 언어-중간언어 / 중간언어-기계어 번역 과정을 분리할 수 있다.

인터프리터는 명령어 인출 -> 의미 분석 -> 실행을 반복하는 프로그램이다. 전체 코드를 모두 기계어로 변환 후 실행하는 컴파일러와 달리 명령어를 읽는 즉시 실행하기 때문에 비교적 빠른 실행 준비 시간과 쉬운 디버깅이라는 장점이 있다. 그러나 한 줄씩 실행하는 것은 비교적 속도가 느리고 최적화가 어렵기 때문에 대표적인 인터프리터 언어인 Python과 Java는 각각의 가상 머신 구조에 맞게 구현된 바이트코드라고 부르는 중간 언어로 컴파일 후 바이트코드를 한줄씩 읽어 인터프리터로 대응되는 기계어 코드를 실행하는 방법을 사용한다. 프로젝트의 mU-Code도 이런 바이트코드의 일종으로 볼 수 있다.

가상 머신은 컴퓨터의 CPU, RAM, 하드디스크 등의 주요 부품들을 소프트웨어적으로 구현하여 사용하는 것이다. 인터프리터에서도 프로그램을 실행하기 위해서 가상 머신 구조를 사용하기도 한다. JVM이 대표적인 인터프리터를 위한 가상 머신이다. 가상 머신은 맞는 바이트코드만 있으면 프로그램을 실행할 수 있기 때문에 고급 언어의 종류는 중요하지 않고 해당 언어가 가상 머신에 맞는 바이트코드로 전환될 수 있는지가 중요하다. kotlin 언어가 자바 바이트코드로 번역되어 JVM에서 실행된다. 이런식으로 고급 언어에서 중간 언어로 바꿔주는 컴파일러와 CPU에 맞는 가상 머신만 있으면, 어떤 언어를 쓰는지 어떤 CPU를 쓰는지 상관없이 실행되기 때문에 이식성과 유연함이 좋다는 장점이 있다. 이 프로젝트에서도 mU-Code를 실행하기 위해서 mU-Code에 맞는 가상 머신 구조를 사용한다.

mU-Code 인터프리터를 구현하며 프로그램의 실행 과정과 가상 머신 구조를 이해하는 것을 목표로 한다. 기존 고급 언어에서는 디버깅 등의 기능으로 실행 과정을 확인할 수 있지만 Low-level에서 명령어 단위로 컴퓨터가 어떻게 동작하는지는 확인하기는 어렵다. 이 프로젝트에서는 중간 언어를 직접 실행하고 분석하면서 이런 Low-level에서의 컴퓨터 동작을 보여준다. 추후 프로젝트를 확장하여 C나 C++ 등의 언어를 mU-Code로 변환해주는 기능을 추가하고 mU-Code에도 명령어를 몇 가지 추가하여 다양한 언어를 실행할 수 있을 것이다. mU-Code 인터프리터를 구현하는것은 컴퓨터 구조와 프로그램 실행에 대한 이해도를 높이고 추후 확장한다면 mU-Code를 이용하는 새로운 나만의 프로그래밍 언어를 한번 만들어 볼 수도 있을 것으로 기대된다.

본 프로젝트의 대표 기능은 명령어를 실행하고 결과를 단계별로 레지스터와 메모리 변화를 시각화하는 것이다. 사용자는 CLI 또는 GUI 환경에서 예제 프로그램을 로드하거나 자체 프로그램을 제작한 뒤 실행 후 내부 상태 변화를 관찰할 수 있다. 또한, 동일한 환경에서 벤치마크를 실행해 프로그램의 성능을 비교할 수 있다. 이러한 사용 시나리오로 본 프로그램은 실행 흐름을 이해하고 분석하는 데 효과적인 학습 도구로 활용된다.

FileRunViewHelp

bubble_sort.uco

uCodeassemble 결과

```

tjp $$END_SORT
ldc 0
str 2 1
$$J nop
lod 2 1
lod 2 2
ldc 1
sub
lod 2 0
sub
ge
tjp $$NEXT_I
% if (a[j] > a[j+1])
lda 2 4
lod 2 1

```

Step 337 of 648

Run

CPU Registers

CS	PC	DS	SS	SP	BP
0	37	104	104	65501	65533

CPU Stack Memory

4	0				
2	0				
	65523				
	0				
	65524				

Labels

Name	Addr	Idx	Block	Offset	Size
\$\$NO_SWAP	75	1	2	3	1
\$\$NEXT_I	80	2	2	4	1
\$\$END_SORT	85	3	2	5	1
\$\$PRINT	87	4	2	6	5

Symbols

Name	Addr	Idx	Block	Offset	Size
\$\$NO_SWAP	75	1	2	3	1
\$\$NEXT_I	80	2	2	4	1
\$\$END_SORT	85	3	2	5	1
\$\$PRINT	87	4	2	6	5

입출력결과 통계lst 파일 출력

입출력

```

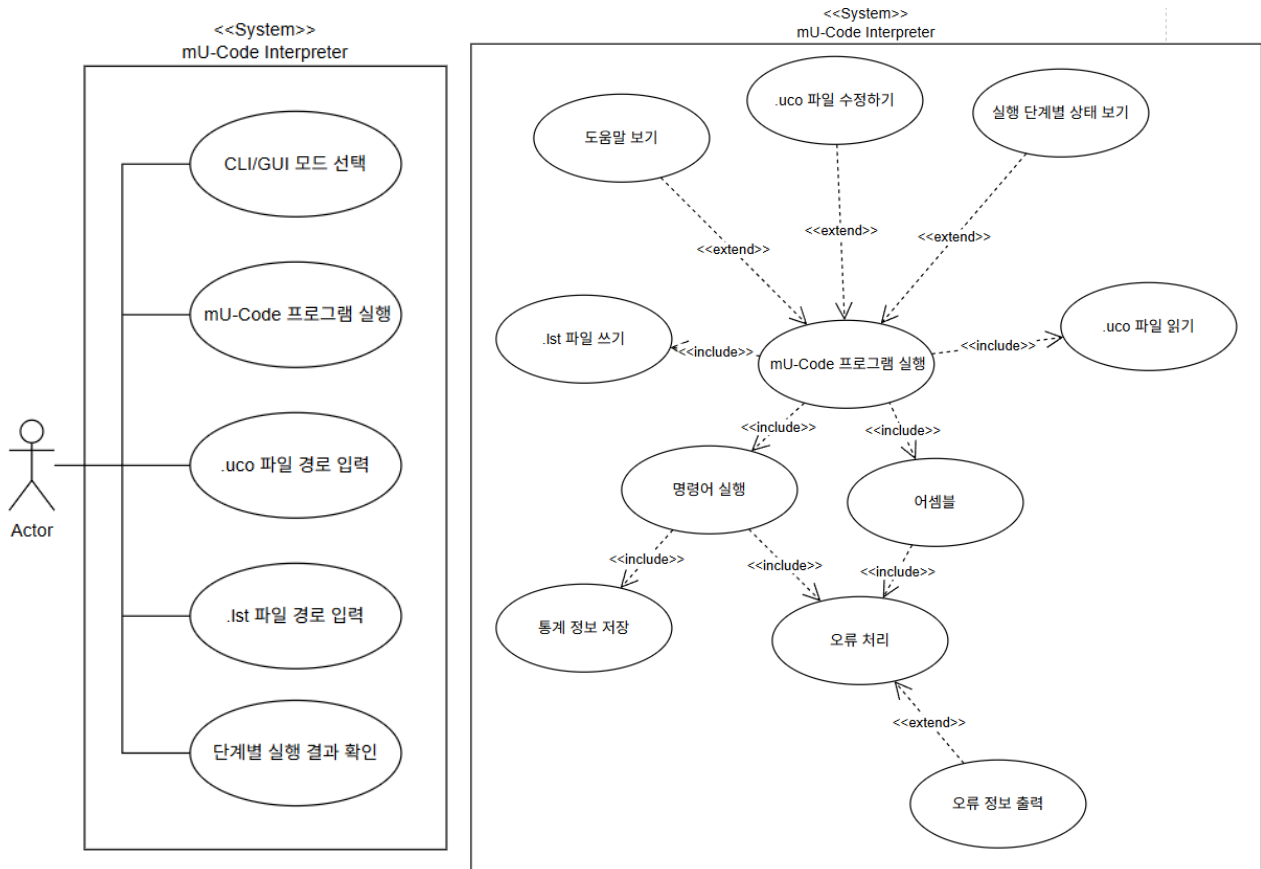
=== 프로그램 시작 ===
>> 4
>> 5
>> 6
>> 2
>> 3
>> 2 3 4 5 6
=== 프로그램 종료 ===

```

2. 요구사항 및 개발환경

2.1 기능 요구사항

[use case diagram]



[사용자 기능 요구사항]

1. .uco 파일 입력
 - 사용자는 CLI 인자로 .uco 파일을 입력하거나 GUI에서 “파일 열기” 기능을 통해 선택할 수 있음
 - 입력: .uco 파일 경로
 - 출력: 선택된 파일 내용이 프로그램 내부에 로드됨
 - 대체 흐름: 잘못된 확장자나 존재하지 않는 경로 입력 시 오류 메시지 출력 후 재입력 요청
2. 코드 실행 및 결과 확인
 - 사용자는 “실행” 버튼을 눌러 전체 코드를 즉시 실행시키며 실행 중 입력이 필요할 경우 CLI 또는 GUI 입력 창을 통해 값을 받음
 - 입력: 실행 명령, 필요시 사용자 입력값
 - 출력: 프로그램 실행 결과(표준 출력 및 상태 정보)
 - 대체 흐름: 실행 중 오류 발생 시 오류 메시지 창 표시, 실행 중단
3. 단계별 실행 결과 확인
 - 프로그램은 실행 완료 후 단계별 상태 변화를 기록하며 사용자는 해당 기록을 확인할 수 있음(실시간 갱신 아님)
 - 입력: 단계 조회 명령(맨 처음/이전/다음/맨 마지막, 임의 단계)
 - 출력: 선택한 단계의 실행 상태 정보

- 대체 흐름: 실행 로그가 존재하지 않거나 손상되었으면 경고 메시지 출력
4. 통계 및 결과 저장
- 실행 결과 및 통계 데이터를 사용자가 지정한 경로로 저장할 수 있음
 - 입력: 저장 경로 및 파일명
 - 출력: .lst 파일 생성
 - 대체 흐름: 저장 실패 시 오류 메시지 출력

[시스템 기능 요구사항]

- 어셈블 처리
 - .uco 파일을 구문분석하여 opcode 및 피연산자 분석, 명령어 테이블 생성 후 실행 코드 영역 구성
 - 입력: .uco 파일
 - 출력: 내부 code 영역에 저장되는 명령어 및 심볼/label 테이블
 - 대체 흐름: 어셈블 오류 발생 시 해당 라인 번호와 원인 로그 출력
- 실행 엔진
 - 어셈블 결과를 기반으로 명령어를 순차 실행하고 메모리, 레지스터를 관리함
 - 입력: 내부 code 영역 명령어
 - 출력: 실행 결과 데이터(출력, 로그)
 - 대체 흐름: 잘못된 명령어나 메모리 접근 시 오류 정보 출력 후 종료
- 오류 처리 및 예외 처리
 - 파일 오류, 구문분석 실패, 실행 중 예외 등 다양한 오류를 감지하고 사용자에게 알림을 전달함
 - 입력: 오류 코드 및 예외 정보
 - 출력: 오류 메시지, 로그 기록
 - 대체 흐름: 오류 발생 시 GUI 경고 창 실행
- GUI 이벤트 처리
 - GTK 신호 기반으로 버튼 클릭, 파일 선택, 종료 이벤트 등을 처리함
 - 입력: 사용자 상호작용 이벤트
 - 출력: 해당 기능 수행 결과

2.2 비기능 요구사항

항목	요구 사항 내용
성능	5개 예제 프로그램 기준 실행 1초 이내
효율성	명령어 주소 인코딩 및 GUI 실행 흐름 최적화로 메모리 사용 효율화
규격/표준	.uco 파일의 코드 형식 준수 (1~10열: 레이블, 11열: 공백, 12열 이후: 명령어/피연산자)
신뢰성	어셈블 및 실행 오류 시 GUI/CLI 모두에서 로그 출력
안전성	잘못된 입력에도 시스템 종료 없이 오류 메시지 출력 후 정상 종료
유지보수성	어셈블러, 실행기, GUI 모듈을 사용하여 분리 구조로 관리
데이터 타입 제한	정수형만 지원
주소 지정 방식	연산 명령: 스택 기반(0주소) 상수/메모리 접근/분기 명령: 피연산자 1개 사용(1주소)
주석 규칙	%로 시작하는 행은 주석으로 인식

개발 환경

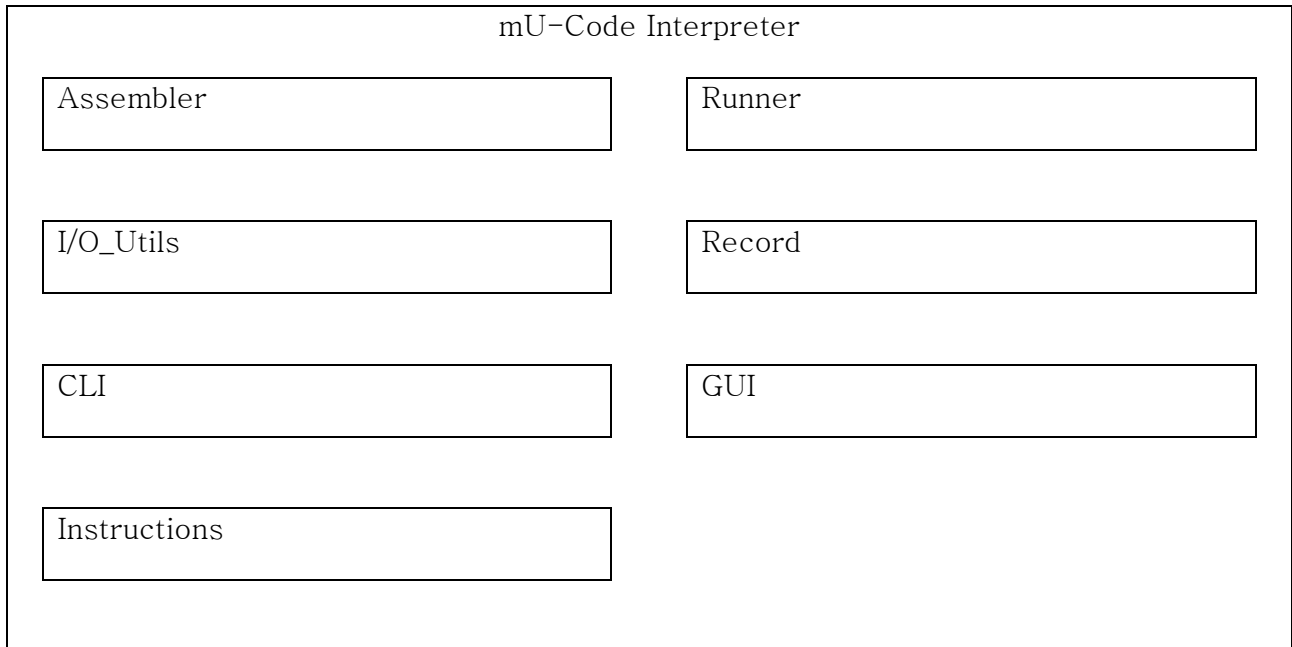
- 개발: C언어, 리눅스, Gtk-3.0, GLib-2.0

-
- 컴파일러: GCC
 - 빌드: Makefile
 - 배포: AppImage
 - 협업: Gtithub
 - OS: Ubuntu
 - 개발 도구: Visual Studio Code

3. 설 계

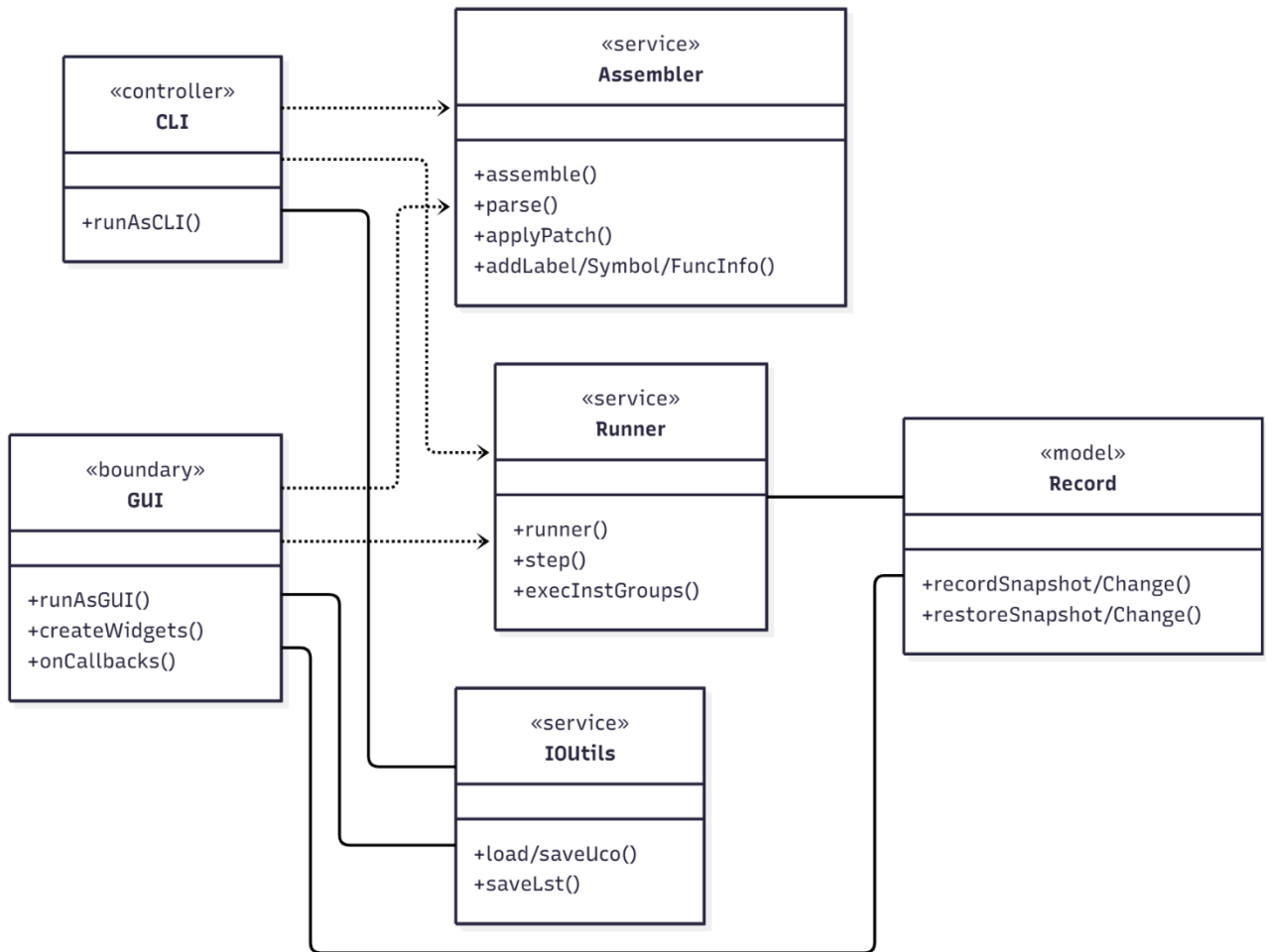
3.1 프로그램 구조 설계

3.1.1 모듈 설계



- Assembler : 입력되는 mU-Code 프로그램을 명령어로 인코딩하고, 실행에 필요한 정보를 추출하는 모듈
- Runner : 어셈블된 mU-Code 프로그램을 실행하는 모듈
- I/O_Utils : .uco 파일을 읽거나 다시 저장, .lst 파일을 생성하는 모듈
- Record : 디버깅을 위한 정보 또는 LST 파일 생성 시 사용하는 정보를 처리하는 모듈
- Instructions : mU-Code의 각 명령어들을 처리하는 모듈
- CLI : CLI 환경으로 실행 시 각종 모듈들을 호출해 프로그램을 실행하는 모듈
- GUI : GUI 환경으로 실행 시 각종 모듈들을 호출해 프로그램을 실행하는 모듈.

3.1.2 모듈간 관계



CLI 또는 GUI 사용자 인터페이스가 각 모듈들을 호출하는 컨트롤러의 역할을 한다. 실행 모듈에서는 명령어 처리를 위해 명령어 모듈과 실행 내용 기록을 위해 기록 모듈을 호출해 사용한다.

3.2 UI 설계

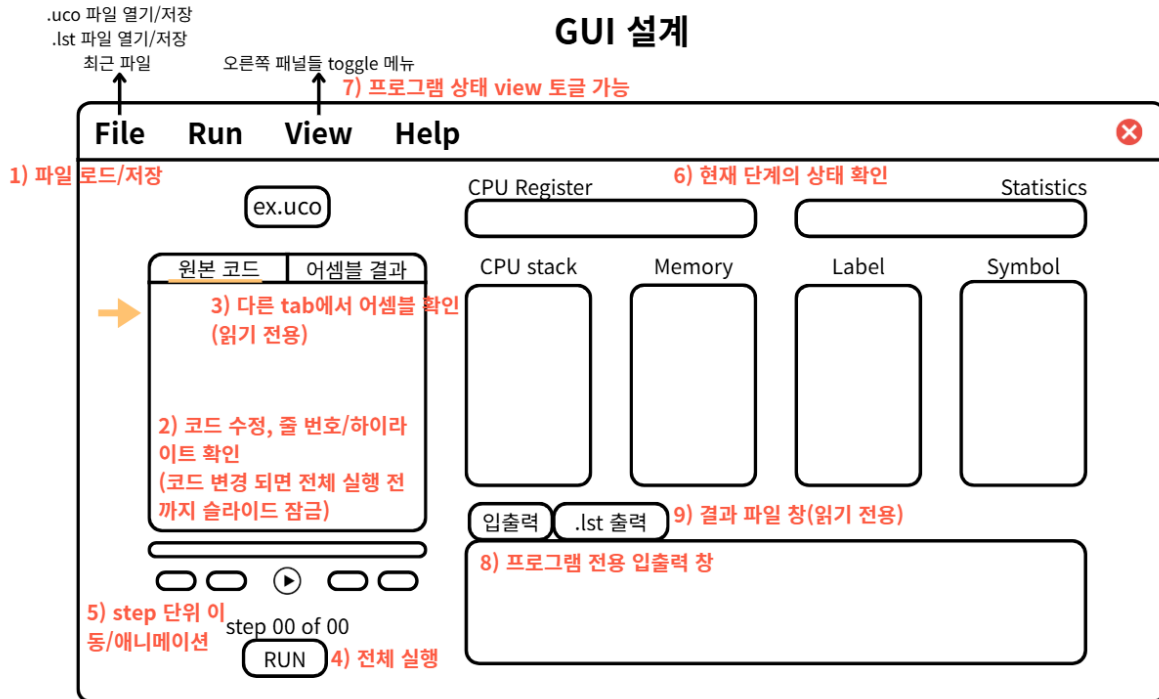
3.2.1 CLI

```

$ ./ucode_interpreter bubble_sort.uco bubble_sort.lst
===== 어셈블 시작 =====
===== 어셈블 완료 =====
===== 실행 시작 =====
131
513
758
11
0
0 11 131 513 758
===== 실행 완료 =====
bubble_sort.lst 에 LST 파일이 생성되었습니다.
$
  
```

사용자는 CLI 환경으로 프로그램 실행 시 인자로 실행하고자 하는 mU-Code 파일과 LST 파일이 생성될 경로를 입력해야한다. 프로그램이 실행되면, 어셈블 단계와 실행 단계가 시작되고 종료된다는 알림 메시지와 LST 파일이 어느 경로에 생성 되었는지에 대한 알림 메시지를 받을 수 있다. 이때 입력된 mU-Code 프로그램에 입출력이 있다면, 실행 단계 중간에 입출력을 할 수 있다. 모든 단계가 완료되면 mU-Code Interpreter는 종료된다.

3.2.2 GUI



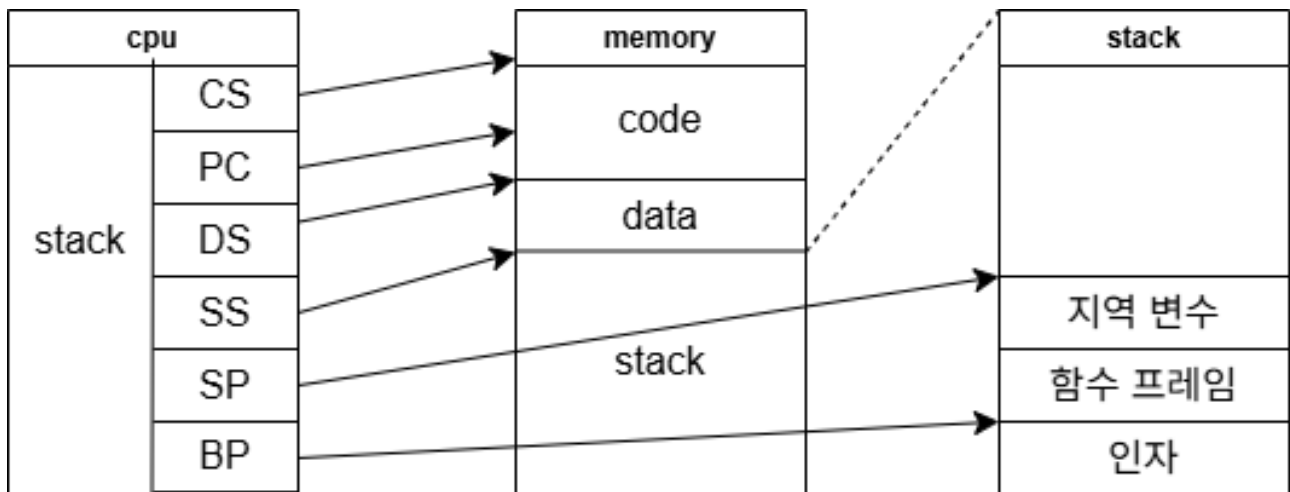
초기 GUI 설계도는 다음과 같은 기능을 지원한다. 사용자는 uco 파일을 열고 저장할 수 있으며, lst 파일은 저장만 가능하다. 최근 파일 목록은 파일 열기 시 GTK 위젯의 최근 파일 기능을 사용하므로 별도로 구현하지 않는다. 코드 뷰에서는 소스 코드를 수정하고 확인할 수 있으며, 실행 버튼으로 프로그램을 실행할 수 있다. 실행 중에는 입출력 패널에서 입력과 출력을 확인할 수 있다. 또한, 실행 단계를 슬라이더와 버튼을 이용해 이동할 수 있다. 실행이 완료되면 어셈블 결과, 가상머신 상태, lst 결과와 통계를 확인할 수 있다. 사용자 안내를 위해 Help 탭에는 명령어 안내, GUI 사용법, 팀 소개를 포함하였다.

3.3 자료 구조 설계

3.3.1 자료 구조 설계 개요

mU-Code Interpreter는 mU-Code 프로그램 실행을 위해 다양한 자료 구조를 사용한다. 주요 자료 구조로 가상 머신 구조가 존재하고 그 외로 필요한 것은 명령어 구조 그리고 어셈블에 필요한 label, symbol, func이다. 또한, GUI 변경 이력 기록에 필요한 change, snapshot도 필요하다.

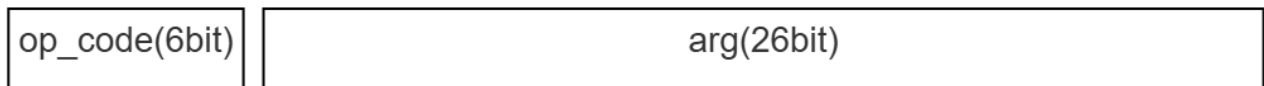
3.3.2 가상 머신 구조



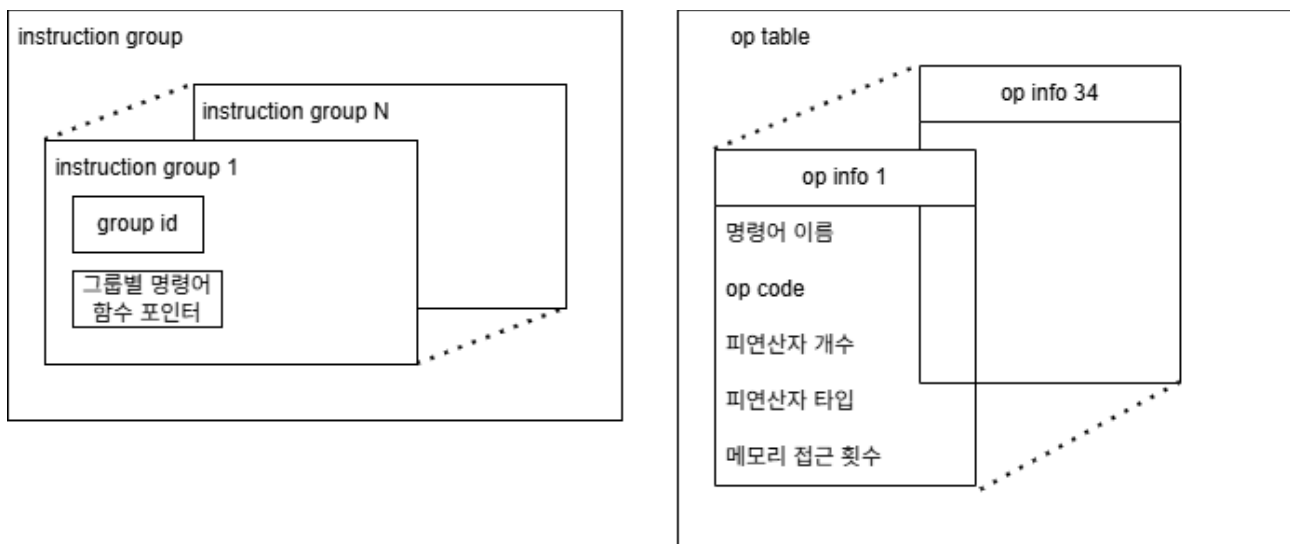
가상 머신의 구조로 크게 CPU와 memory로 구성된다. CPU 내부에서는 CPU stack과 함께 6가지 레지스터가 존재하고, memory는 code, data, stack 영역을 가진다. code 영역은 기계어가 저장되며 data 영역은 전역 변수가 저장된다. 그리고 stack은 지역 변수가 저장되는 곳으로 인자, 함수 프레임, 지역 변수 순으로 쌓이는 구조이다.

3.3.3 세부 자료 구조

3.3.3.1 명령어 구조



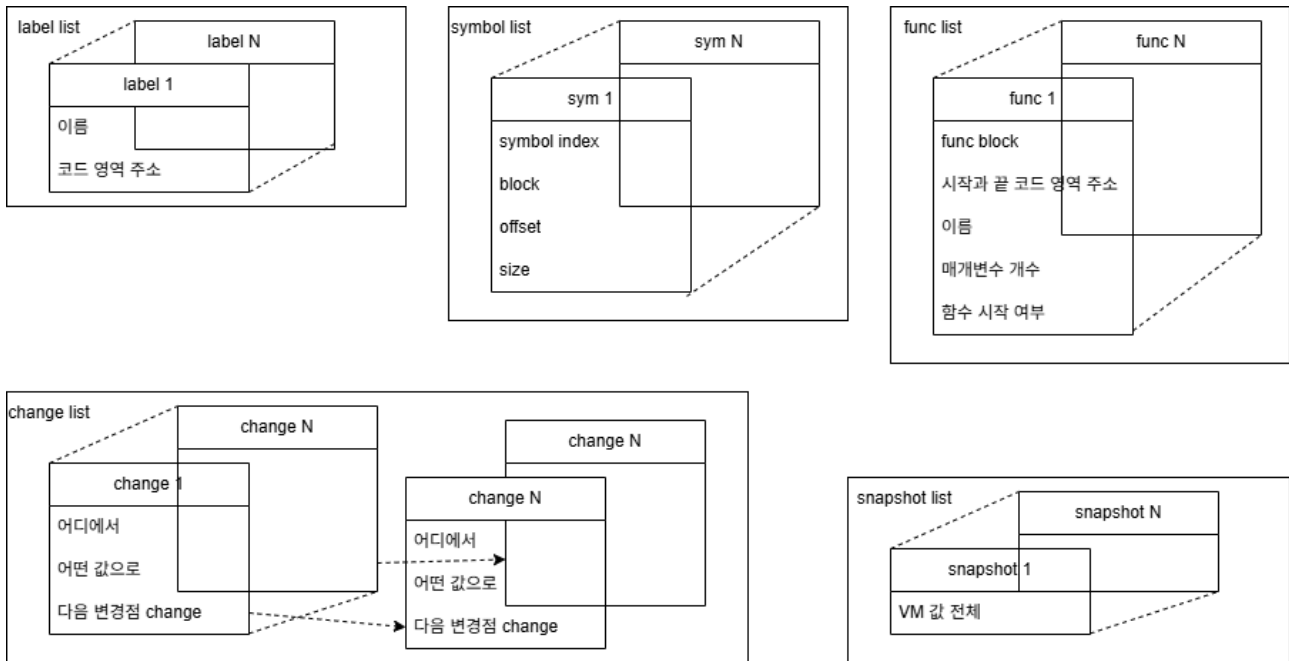
명령어 형식은 32bit 정수형으로 상위 6비트는 opcode, 하위 26비트는 피연산자로 설계하였다. 또한, 명령어 내부에서도 명령어 기능 단위로 분류하기 위해 opcode의 상위 3비트는 명령어 그룹을, 하위 3비트는 그룹 내 인덱스로 설계하였다.



위처럼 설계된 명령어 형식에 따라 instruction group은 각각 그룹화된 함수 포인터를 가진다. 그리고 명령어 별로 처리 로직을 쉽게 하려고 명령어 34개를 넣은 명령어 정보를 table로 관리하도록 설계하였다. 명령어 정보로는 이름, opcode, 검증용으로 필요한 피연산자 개수와

타입, 통계를 산출하기 위한 명령어별 메모리 접근 횟수가 있다.

3.3.3.2 label, sym, func, change, snapshot



상단은 어셈블 과정에서 필요한 label, symbol, func 자료구조이다. 라벨은 라벨 등록에 필요하며 라벨 등록 전에 나오는 라벨을 처리하기 위한 patch 전용 자료구조가 존재한다. 심볼은 변수 등록에 필요하며 따로 patch 자료 구조가 존재하지 않는다. 또한, func에는 매개변수를 찾아내는 데 필요하며 함수 등록 전에 나오는 함수를 처리하기 위한 patch 자료 구조가 존재한다. 이 자료구조는 proc로 시작하면 ret을 하는지 검증까지 하므로 시작과 끝 코드 영역 주소를 추가로 저장한다. 또한, 변수의 offset 값을 조절할 때 함수랑 연결되어야 하므로 block을 추가로 기재하여 인자 개수를 찾을 수 있도록 설계하였다.

하단은 GUI 상태 저장용으로 필요한 change, snapshot 자료구조이다. change list는 step 별 change 헤드 포인터 배열이다. 각 change는 어떤 변수가 어떤 값으로 변경되는지 값을 저장하며 그리고 같은 step에 발생한 다른 변경 점을 연결리스트로 이어서 저장하도록 설계하였다. 다음으로 snapshot은 N 단계마다 가상 머신에서 사용하는 변수 전체를 저장한다. 이렇게 설계한 이유는 전체 변수를 저장하면 메모리 효율이 높지 않고 그렇다고 변경된 점만 기록해 두면 메모리 측면에서는 좋겠지만 변경 점을 적용하는 데 오래 걸리기 때문에 절충안을 택한 것이다.

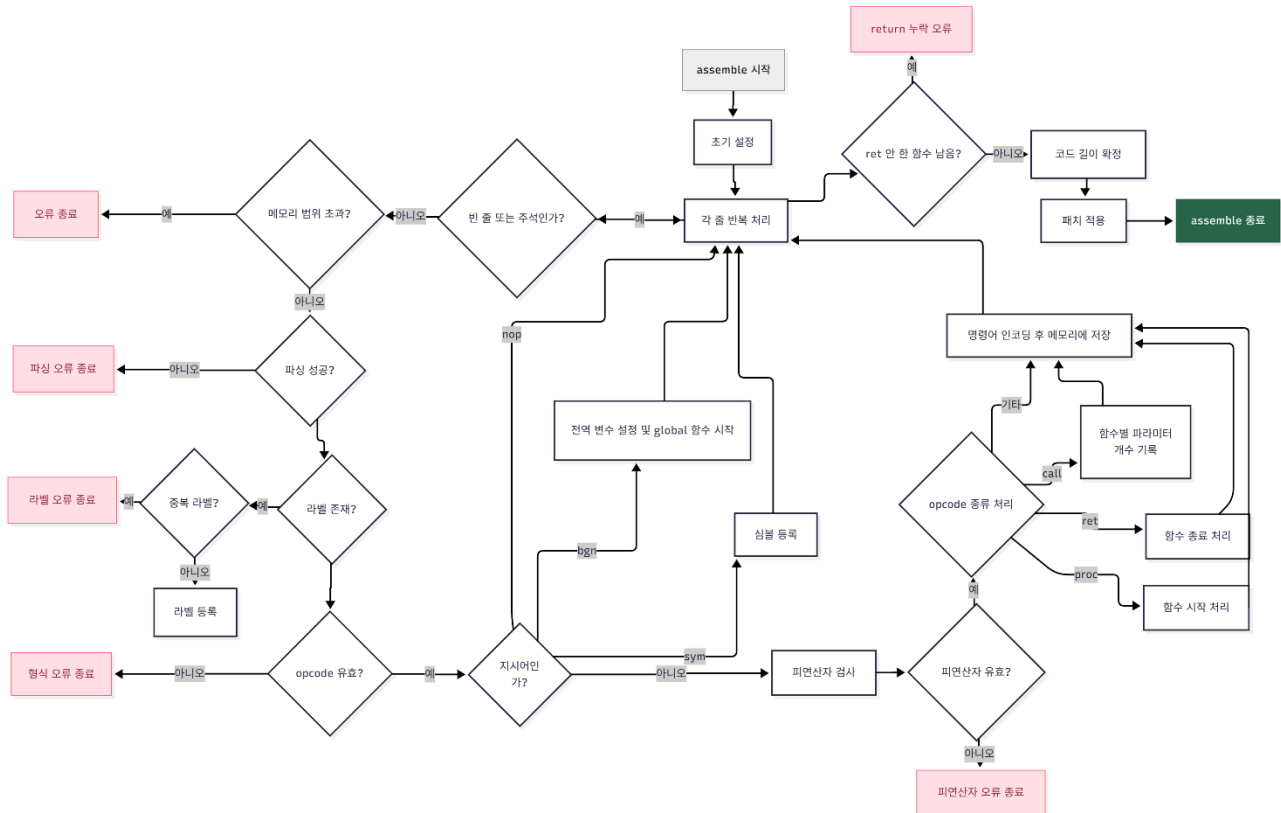
3.4 알고리즘 설계

3.4.1 알고리즘 설계 개요

mU-Code Interpreter에서는 mU-Code 프로그램 실행 및 디버깅을 위한 다양한 알고리즘이 있다. 각 알고리즘은 모듈 간의 독립성을 해치지 않는 선에서 최대한 좋은 성능을 보이도록 노력하며 설계했다.

3.4.2 주요 알고리즘 flowchart/sequence diagram

3.4.2.1 어셈블 단계



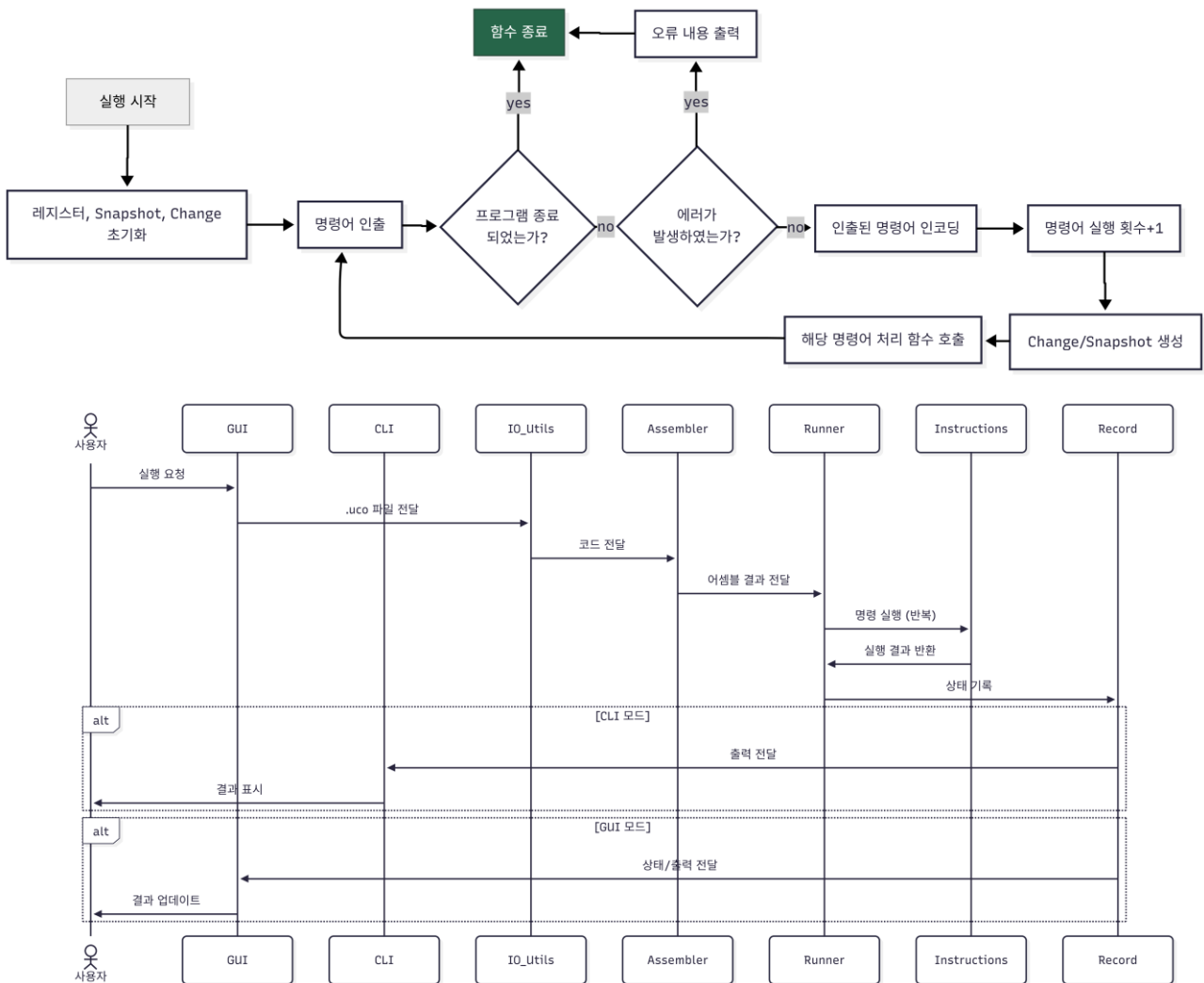
어셈블 함수는 입력으로 받은 각 라인을 반복 처리하며, 빈 줄이나 주석은 무시한다. 각 줄에 대해 메모리 범위, parse, label, symbol(변수), opcode, 피연산자, proc/ret(함수 짝) 오류를 점검한다. 처리 과정은 다음과 같다. 먼저 각 줄을 parse 하여 label, opcode, 피연산자로 분리하고, opcode 종류에 따라 적절히 처리한 후 필요한 패치를 적용한다. 모든 줄을 처리하면 어셈블 함수가 종료된다.

Opcode 처리 방식은 다음과 같다.

- bgn: 전역 변수를 설정하고 global 함수로 등록
- sym: 변수 등록
- proc: 함수 시작 등록
- ret/end: 현재 함수 종료 처리
- call: 기록된 파라미터 개수를 함수에 저장
- ldp: 파라미터 개수 초기화
- push: 파라미터 개수 추가
- 그 외 명령어: 인코딩 후 메모리의 code 영역에 저장

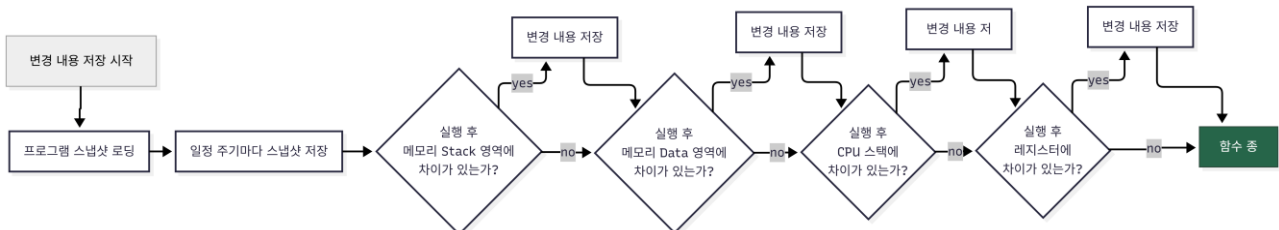
마지막으로 label과 symbol의 offset 값, 함수 정보를 패치하여 모든 처리를 완료한다.

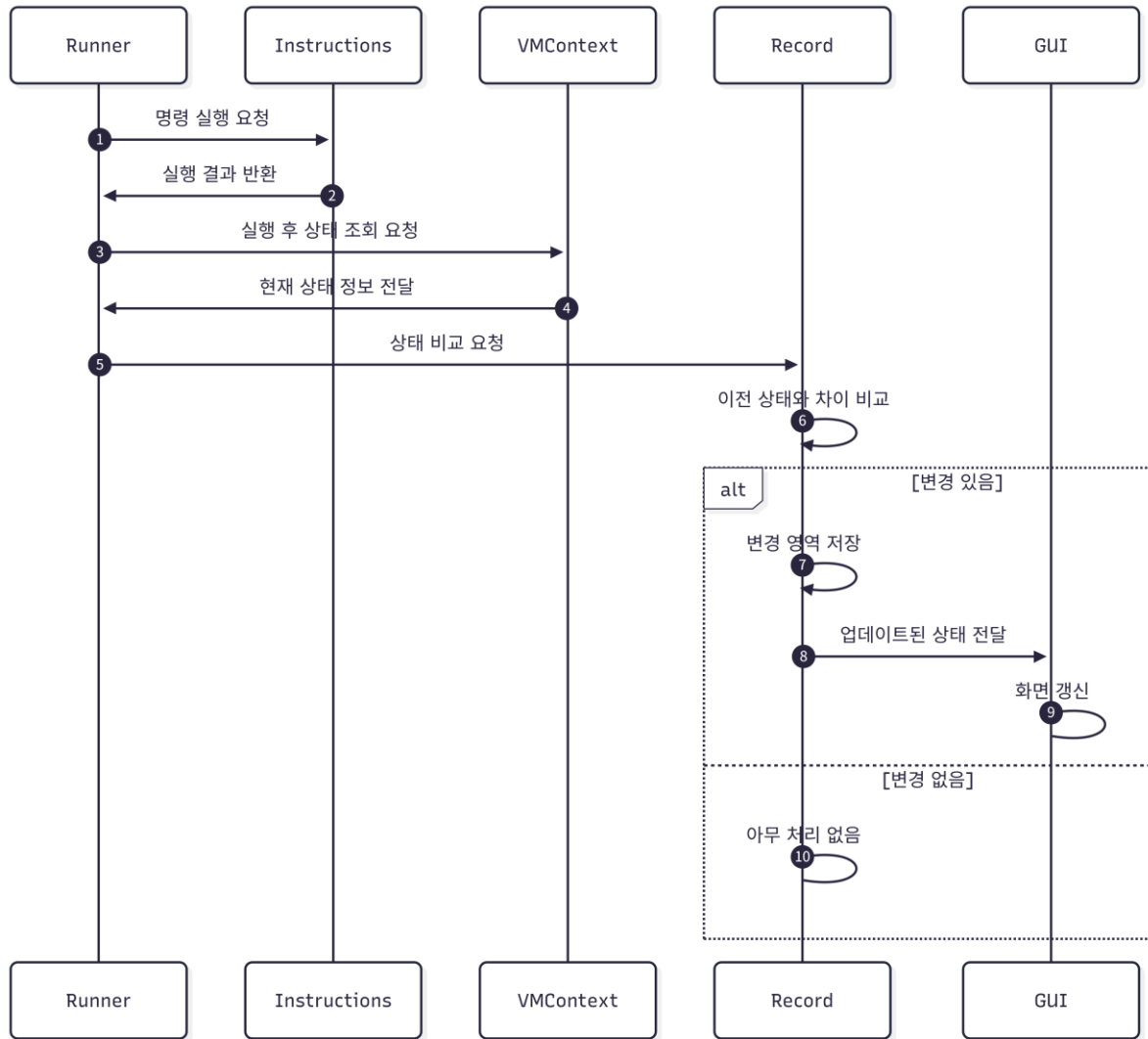
3.4.2.2 실행 단계



실행 단계에서는 어셈블 단계에서 인코딩되어 메모리에 저장된 명령어를 인출해 차례대로 실행한다. 먼저 실행에 관련된 레지스터들과 디버깅에 필요한 Snapshot, Change 등을 초기화한 후 인출, 해독, 실행 과정을 반복한다. 이때 실행된 명령어는 통계에 반영된다.

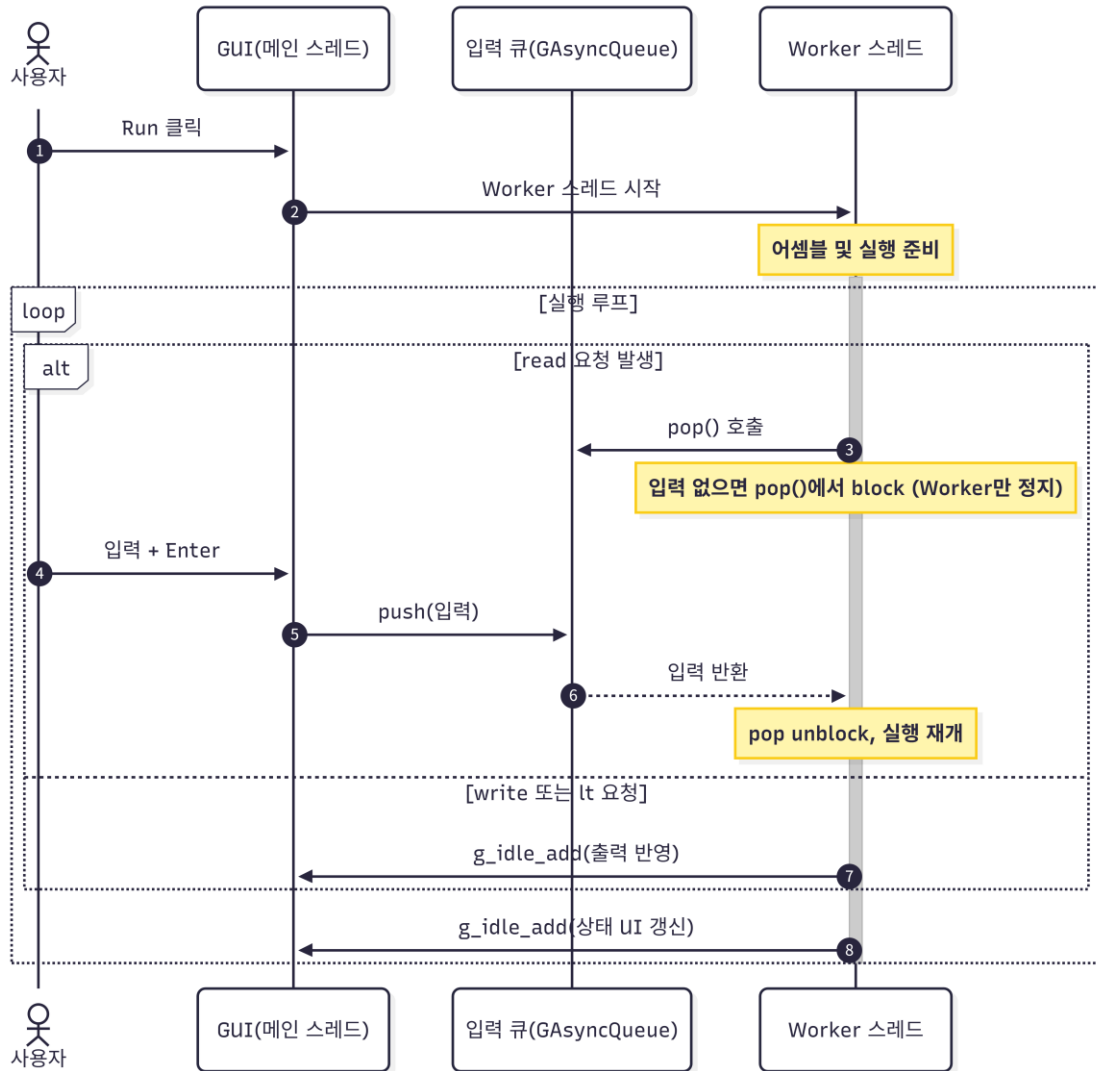
3.4.2.3 Change/Snapshot 생성





Change/Snapshot 생성 단계에서는 명령어 실행 전과 후를 비교해 변경된 내용을 Record 모듈에 저장한다. 저장할 때는 연결리스트의 형태로 변경 내용들이 저장된다. 비교할 때는 Record 내부의 별도 스냅샷을 관리하며 VMContext와 일정 부분을 비교해 변경 내용을 발견한다. 또한 프로그램 실행 일정 주기마다 전체 스냅샷을 저장해 추후 실행 단계별 상태 확인 시에 사용한다.

3.4.2.4 GUI 비동기 알고리즘



메인 스레드는 Run 버튼 클릭 시 비동기 큐를 초기화하고 필요한 준비 작업을 수행한다. 이후 워커 스레드를 시작하여 소스 코드를 어셈블하고 실행하는 작업을 맡긴다. 실행 과정에서 발생하는 GUI 업데이트는 모두 메인 스레드에서 처리되도록 설계되어 있다.

메시지 비동기 처리 방식은 다음과 같다. 메인 스레드는 사용자의 입력을 받을 때마다 비동기 큐에 push만 수행한다. 워커 스레드는 실행 중 필요한 시점에 pop을 호출하며, pop 이후에는 큐에 새로운 데이터가 push 될 때까지 블로킹 상태로 대기한다. Push가 발생하면 pop이 즉시 깨어나 데이터를 가져가고, 이 과정은 큐 자료구조이므로 FIFO 순서로 처리된다.

4. 구현

4.1 자료 구조 구현

vm_context.h

```
typedef struct {
    int cs;
    int pc;
```

```

int ds;
int ss;
int sp;
int bp;
int code_len; // 코드 영역 길이
int g_var_cnt; // 전역변수 개수
int prev_pc; // 현재 실행 중 pc 라인

Labellist label_list; // 라벨
PatchList patch_list; // 패치할 라벨
SymbolList symbol_list; // 변수 정보
FuncList func_list; // 함수 매개변수 정보
CallPatchList call_patch_list; // 매개변수 패치할 함수

unsigned int flags; // 오류 플래그 (실행기)
int memory[INIT_MEMORY_SIZE]; // memory stack
Stack cpu_stack; // cpu stack
InstructionGroup inst_group[TOTAL_OPCODE_GROUPS]; // 함수 포인터 매핑
Changelist changes; // 변경점 기록
SnapshotList snapshot_list; // N 단계 스냅샷 기록
Statistics stat; // 통계
} VMContext;

```

설계 그림대로 각 자료구조를 포함했다. 가상 머신 구조와 함께 어셈블, 명령어 그룹, 통계, 기록과 관련된 구조체를 한꺼번에 들고 있도록 구현하였다.

types.h

```

// 라벨
typedef struct {
    char name[MAX_LABEL_LEN];
    int addr;
} Label;
typedef struct {
    Label *labels;
    int count;
    int capacity;
} Labellist;
typedef struct {
    int code_idx;
    int src_idx;
    char label_name[MAX_LABEL_LEN];
} Patch;
typedef struct {
    Patch *patches;
    int count;
    int capacity;
} PatchList;

// 심볼
typedef struct {
    int index;
    int block;
    int offset;
    int size;
} Symbol;
typedef struct {

```

```

    Symbol *symbols;
    int count;
    int capacity;
} SymbolList;

// 함수 매개변수 처리
typedef struct {
    int func_block;
    int start_addr;
    int end_addr;
    char name[MAX_LABEL_LEN];
    int param_cnt;
    int is_start;
} FuncInfo;
typedef struct {
    FuncInfo *items;
    int count;
    int capacity;
} FuncList;
typedef struct {
    char func_name[MAX_LABEL_LEN];
    int param_cnt;
} CallPatch;
typedef struct {
    CallPatch *patches;
    int count;
    int capacity;
} CallPatchList;

// 변경점 연결리스트
typedef struct Change {
    int hardware_num;
    int addr;
    int new_value;
    struct Change *next;
} Change;
typedef struct {
    Change **change_list;
    int list_count;
    int list_size;
} ChangeList;

// 스냅샷 리스트
typedef struct {
    int memory[INIT_MEMORY_SIZE];
    int cpu_stack[INIT_CPU_STACK_CAPACITY];
    int cpu_top;
    int cs;
    int pc;
    int ds;
    int ss;
    int sp;
    int bp;
    int flags;
    int prev_pc; // 현재 실행 중
} Snapshot;
typedef struct {
    Snapshot *snapshot_list;

```

```

    int snapshot_count;
    int snapshot_capacity;
} SnapshotList;

// 명령어 그룹
typedef struct {
    int group_id;
    void (*execInst)(int);
} InstructionGroup;

// 통계
typedef struct {
    int inst_use_count[OPCODE_MAX];
    int inst_run_count[OPCODE_MAX];
    int memory_access_count;
} Statistics;

// cpu 스택
typedef struct {
    int capacity;
    int top;
    int *items;
} Stack;

```

context 내부에서 사용한 세부 구조체들이다.

instruction.h

```

typedef enum {
    OPERAND_NONE = 0,
    OPERAND_NUMBER,
    OPERAND_LABEL,
    OPERAND_BLOCK_OFFSET
} OperandType;

typedef struct {
    const char *name;           // 명령어 이름
    int opcode;                 // opcode
    int operand_count;          // 피연산자 개수
    OperandType operand_type;   // 피연산자 타입
    int mem_access;             // 명령어별 메모리 접근 횟수
} OpInfo;
// == instruction.c == //
static const OpInfo op_table[] = { // 생략 };

```

명령어 정보는 따로 분리하여 구현하였다.

4.2 핵심 소스 코드

4.2.1 어셈블 코드

```

ErrorResult assemble(char **lines, int line_count) {
    // 각 라인 처리
    for (int i = 0; i < line_count; i++) {
        char *line = lines[i];
        if (*line_ptr == '%' || !*line_ptr)
            continue; // 주석, 빈줄 제외
        // 코드 파싱
        char label[MAX_LABEL_LEN];

```

```

char opcode[MAX_OP_LEN];
char *operands[MAX_OPERANDS];
int operand_count = 0;
parseLine(line, label, opcode, operands, &operand_count);
// 라벨 추가
if (label[0])
    addLabel(label, addr);
// opcode 확인
const OpInfo *info = findOpInfoByName(opcode);
// 어셈블러 전용 opcode 처리
if (info->opcode < 0) {
    if (strcmp(info->name, "bgn") == 0) {
        ctx->g_var_cnt = atoi(operands[0]);
        current_func = addFunc("global", addr);
        current_func->func_block = 1;
        continue;
    } else if (strcmp(info->name, "sym") == 0) {
        int block = atoi(operands[0]);
        int offset = atoi(operands[1]);
        int size = atoi(operands[2]);
        addSymbol(block, offset, size);
        const char *name = findLabelByAddr(addr - 1);
        FuncInfo *func = findFuncByName(name);
        func->func_block = block;
        continue;
    } else if (strcmp(info->name, "nop") == 0) {
        continue;
    }
}
// operands 확인
int operand_val = 0;
validOperands(info, operands, operand_count, &operand_val, addr, i + 1);
// 함수 인자(매개변수) 개수 처리
switch (info->opcode) {
case OP_PROC: {
    current_func = addFunc(label, addr);
    CallPatch *patch = findCallPatchByName(current_func->name);
    operand_val -= patch->param_cnt;
    break;
}
case OP_RET: {
    current_func->end_addr = addr;
    current_func->is_start = 0;
    CallPatch *patch = findCallPatchByName(current_func->name);
    current_func->param_cnt = patch->param_cnt;
    operand_val = patch->param_cnt;
    param_cnt = 0;
    current_func = NULL;
    break;
}
case OP_LDP:
    param_cnt = 0;
    break;
case OP_PUSH:
    if (current_func && param_cnt < MAX_ARGS) {
        param_cnt++;
    }
    break;
}

```

```

case OP_CALL: {
    if (isSystemLabel(operands[0])) {
        param_cnt = 0;
        break;
    }
    FuncInfo *func_info = findFuncByName(operands[0]);
    if (func_info) {
        func_info->param_cnt = param_cnt;
    } else {
        addCallPatch(operands[0], param_cnt);
    }
    param_cnt = 0;
    break;
}
}
ctx->memory[addr++] = encodeInst(info->opcode, operand_val);
}
ctx->code_len = addr;
applyPatches();
applyCallPatch();
applySymbolOffset();
return ok;
}

```

원본 코드 중 오류 처리 부분과 불필요한 부분은 제외하고 첨부하였다.

4.2.2 실행 엔진 코드

```

// === runner.c === //
void step(void) {
    VMContext *ctx = getVMContext();
    int inst = ctx->memory[ctx->pc++];
    int group_code;
    int g_idx;
    decodeInst(inst, &group_code, &g_idx, NULL);
    int opcode = group_code * 10 + g_idx;
    ctx->stat.inst_run_count[opcode]++;
    const OpInfo *op = findOpInfoByOpcode(opcode);
    if (op)
        ctx->stat.memory_access_count += op->mem_access;

    ctx->inst_group[group_code].execInst(inst);
    return;
}

ErrorResult runner() {
    ErrorResult ok = {ERR_SRC_NONE, 0, -1};
    VMContext *ctx = getVMContext();
    readyToRun();
    initSnapshot();
    ctx->prev_pc = 0;
    while (1) {
        if (ctx->bp == -1) {
            printf("\n[INFO] runner stopped\n");
            return ok;
        }
        if (ctx->flags != 0) {
            printRunError(ctx->source_map.line[ctx->prev_pc]);
            return (ErrorResult){ERR_SRC_RUNNER, ctx->flags,
                                ctx->source_map.line[ctx->prev_pc]};
        }
    }
}

```

```

    }
    ctx->prev_pc = ctx->pc;
    step();
    saveChanges();
}
}
// === runner_instruction_group.c === //
void execInstGroup0(int inst) {
    int arg;
    int opcode;
    decodeInst(inst, NULL, &opcode, &arg);
    switch (opcode) {
    case 0:
        proc(arg);
        break;
    case 1:
        ret(arg);
        break;
    case 2:
        ldp(arg);
        break;
    case 3:
        push(arg);
        break;
    case 4:
        call(arg);
        break;
    }
    return;
}
}

```

위 소스 코드는 실행 엔진 중 핵심이 되는 코드이다. Step 함수는 명령어 한 개에 대하여 인출 → 해독 → 실행과정을 진행 하는 함수이다. 가상 머신 메모리에서 pc 값을 가지고 명령어를 인출해 명령어를 해독한 뒤, 통계 정보에 이를 반영하고 명령어를 처리할 함수를 호출하게 된다.

Runner 함수는 실행 단계를 진행하는 함수로, 실행에 필요한 레지스터와 기록할 스냅샷을 초기화 한 후 종료 조건인 BP == -1, 에러 플래그 세트가 될때까지 step 함수 호출과 변경 내용 기록을 반복한다.

execInstGroup0은 명령어를 처리하는 함수 직전 단계로 명령어를 그룹으로 나눠두고 한 그룹의 명령어들로 매핑해준다. 이 함수 외에도 그룹별로 execInstGroup1, execInstGroup2 등의 함수가 존재한다.

4.2.3 GUI 스레드 및 입출력 비동기 코드

```

// === vm_runner.c === //
// run 버튼 클릭시 콜백 함수
void onRun(GtkButton *button) {
    // 프로그램 시작 전 작업 처리 생략
    io_ctx->input_queue = g_async_queue_new(); // 비동기 큐 초기화
    startWorker(wd);                          // 워커 스레드 시작
}

// === worker_thread.c === //
static gpointer runWorkerThread(gpointer arg) {
    // 작업 데이터(원본 코드) 들고 일하기
    WorkerData *wd = (WorkerData *)arg;
}

```

```

assemble(wd->lines, wd->line_count);
runner();
// 작업 완료 후 UI 업데이트
return NULL;
}
void startWorker(WorkerData *wd) {
    g_thread_new("worker", runWorkerThread, wd);
}

// === io.c === //
// GUI 에서 입력된 한 줄을 int 포인터로 변환하여 비동기 큐에 넣는 함수
static void handleLineInput(IOContext *io_ctx, const gchar *line) {
    int *ptr = g_new(int, 1);
    *ptr = atoi(trimmed);
    g_async_queue_push(io_ctx->input_queue, ptr);
}
// 키입력 콜백 함수
gboolean onIOKeyPress(GtkWidget *widget, GdkEventKey *event,
                      gpointer user_data) {
    // 다른 이벤트 처리 생략
    if (event->keyval == GDK_KEY_Return) {
        // 엔터 감지하면 현재 줄 텍스트 가져온 후 prefix 검사
        handleLineInput(io_ctx, user_input); // 여기서 큐에 넣음
        return TRUE; // 키 입력 허용
    }
    return FALSE;
}
// worker thread 내부 runner 함수 내부에서 호출되는 IO 함수들
int guiIoRead() {
    // int 포인터로 넣은 값을 꺼내서 반환
    int *ptr = g_async_queue_pop(io_ctx->input_queue);
    int value = *ptr;
    g_free(ptr);
    return value;
}
void guiIoWrite(const char *data) {
    // 메인 스레드 전용 wrapper 함수로 gui 업데이트 작업 수행
}
void guiIoLf() {
    // 메인 스레드 전용 wrapper 함수로 gui 업데이트 작업 수행
}

```

스레드 생성 시 `g_thread_new()` 함수를 사용하였고 스레드 종료는 `return NULL`로 구현했다. 메시지 큐는 `g_async_queue_new()`로 초기화하고 메인 스레드에서 `g_async_queue_push()`로 int 포인터 값을 삽입한다. 그리고 `g_async_queue_pop()`으로 워커 스레드에서 pop을 하여 값을 전달한다.

4.3 실행 화면

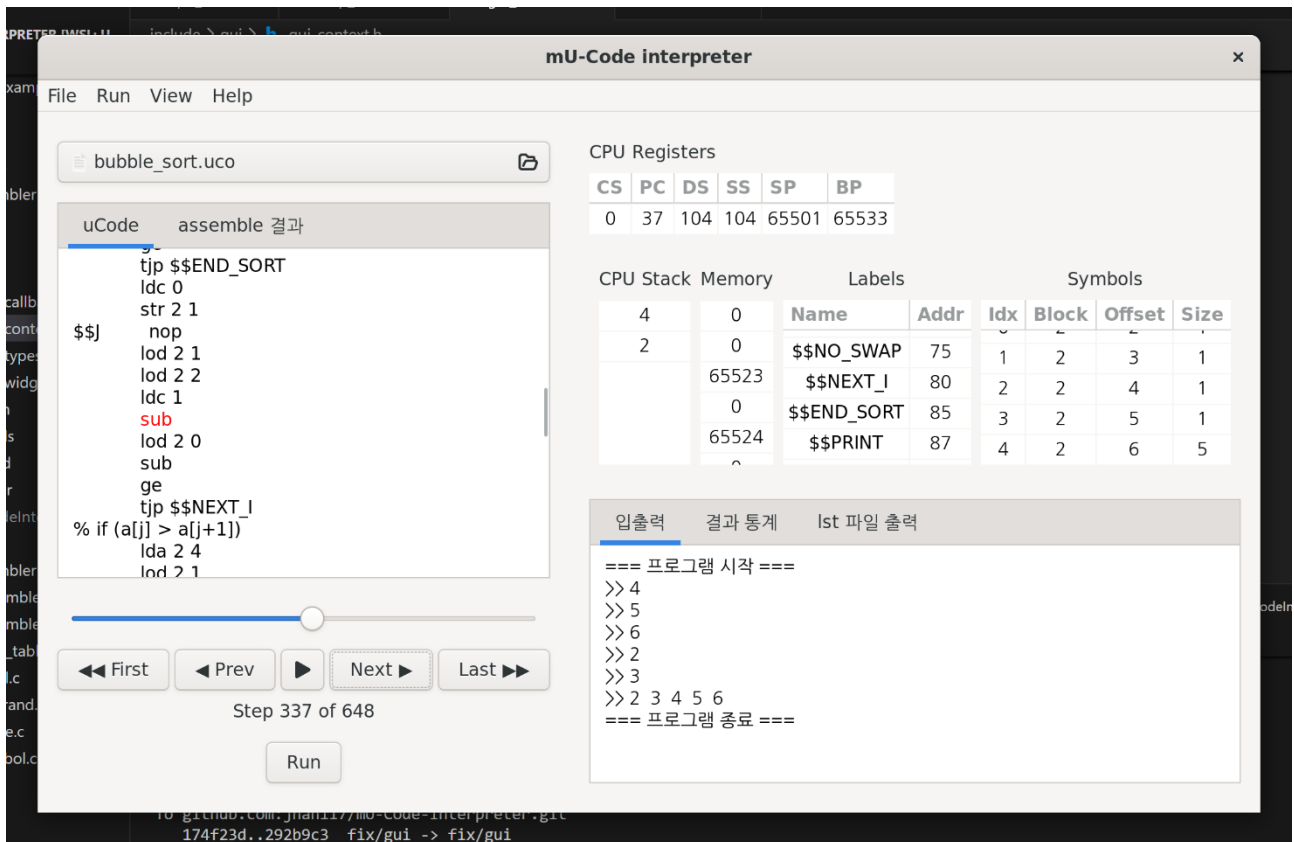
CLI 실행 화면


```

dlrkdtr@DESKTOP-SM810A9:~/SWP3/swp-3$ ./build/ucode_interpreter assets/uco_example/bubble_sort.uco test.lst
===== 어셈블 시작 =====
===== 어셈블 완료 =====
===== 실행 시작 =====
4
5
6
2
3
2 3 4 5 6
[INFO] runner stopped
===== 실행 완료 =====
test.lst 에 LST 파일이 생성되었습니다.

```

GUI 실행 화면



LST 파일

```
dlrkdjtr@DESKTOP-SM81OA9: × + ▾
==== 원본 코드 ====      Encoded      Opcode Operand
[0001] % int main() {
[0002] %   int a[5];
[0003] %   int n = 5;
[0004] %   int i = 0, j;
[0005] %
[0006] %   // input
[0007] %   while (i < n) {
[0008] %       read(&a[i]);
[0009] %       i++;
[0010] %   }
[0011] %
[0012] %   // bubble sort
[0013] %   for (i = 0; i < n - 1; i++) {
[0014] %       for (j = 0; j < n - 1 - i; j++) {
[0015] %           if (a[j] > a[j+1]) {
[0016] %               int temp = a[j];
[0017] %               a[j] = a[j+1];
[0018] %               a[j+1] = temp;
[0019] %           }
[0020] %       }
[0021] %   }
[0022] %
[0023] %   // print
[0024] %   for (i = 0; i < n; i++)
[0025] %       write(a[i]);
[0026] % }
[0027]
[0028] % 프로그램 시작
[0029]     bgn 0
[0030]     ldp          0x08000000 2      0
[0031]     call main    0x10000003 4      3
[0032]     end          0x04000000 1      0
[0033]
[0034] % main
[0035] main      proc 20      0x00000014 0      20
[0036]
[0037] % i, j, n, temp, a[5]
[0038]     sym 2 0 1
[0039]     sym 2 1 1
[0040]     sym 2 2 1
[0041]     sym 2 3 1
[0042]     sym 2 4 5
[0043]
[0044] % n = 5
[0045]     ldc 5        0x48000005 22      5
[0046]     str 2 2      0x4C000002 23      2
[0047]
```

```

[0182]          ret          0x04000000 1      0

=====실행 결과=====
2 3 4 5 6

=====명령어 사용 횟수=====
proc  = 1      ret   = 2      ldp   = 3
push  = 2      call  = 3      ujp   = 4
tjp   = 3      fjp   = 2      lod   = 22
lda   = 8      ldc   = 14     str   = 10
ldi   = 5      sti   = 2      gt    = 1
lt    = 1      ge    = 3      le    = 0
eq    = 0      ne    = 0      and   = 0
or    = 0      add   = 7      sub   = 11
mult  = 0      div   = 0      mod   = 0
not   = 0      neg   = 0      dup   = 0

=====명령어 실행 횟수=====
proc  = 1      ret   = 2      ldp   = 11
push  = 10     call  = 11     ujp   = 24
tjp   = 25     fjp   = 16     lod   = 160
lda   = 54     ldc   = 73     str   = 38
ldi   = 37     sti   = 12     gt    = 10
lt    = 6      ge    = 25     le    = 0
eq    = 0      ne    = 0      and   = 0
or    = 0      add   = 46     sub   = 87
mult  = 0      div   = 0      mod   = 0
not   = 0      neg   = 0      dup   = 0

=====기타 통계=====
메모리 접근 횟수 = 337

```

4.4 결과 분석

4.4.1 실험 조건

- CPU : Intel® Core™ Ultra 5 228V
- RAM : 16GB (16 122 076 kB)
- OS : Ubuntu 24.04.3 LTS (WSL2)
- C Compiler : GCC 13.3.0
- Python : Python 3.12.3
- Java : OpenJDK 21.0.9

[mU-Code Interpreter 설정]

- 모든 텍스트 출력 비활성화
- LST 파일 생성 비활성화
- Snapshot/Change 생성 비활성화

[테스트 코드]

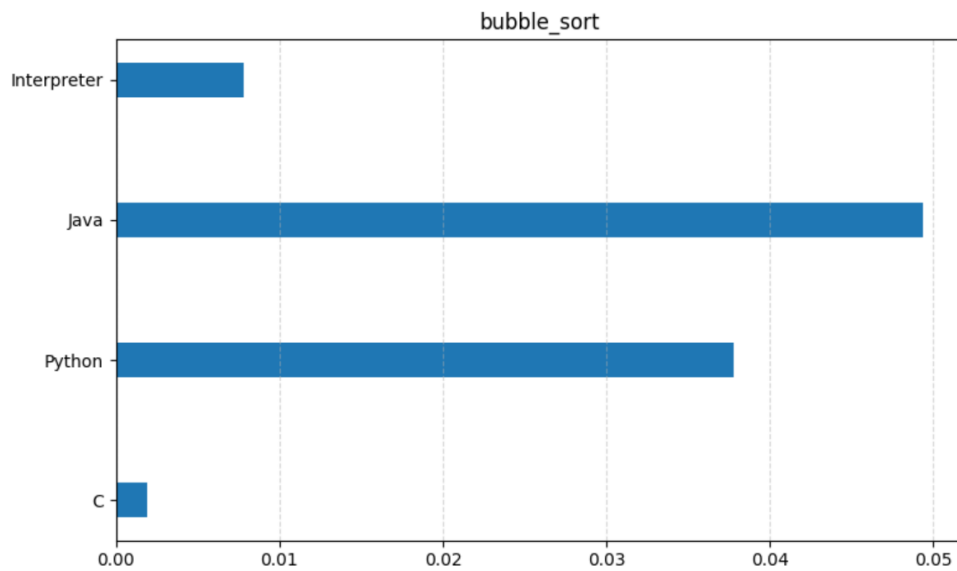
- 버블 정렬 : 길이가 1000이고 값이 역순으로 초기화된 배열 정렬
- 이진 트리 DFS : 노드 수가 10000개인 배열 기반 이진트리 깊이 우선 탐색
- 피보나치 수열 : 재귀 호출을 통해 30번째 피보나치 수 탐색

[측정 방법]

- 실행 시간 : 쉘 스크립트 date +%s%N 실행 전/후 시간 비교
- 메모리 사용량 : /usr/bin/time -v 의 Max Resident set size 값 사용

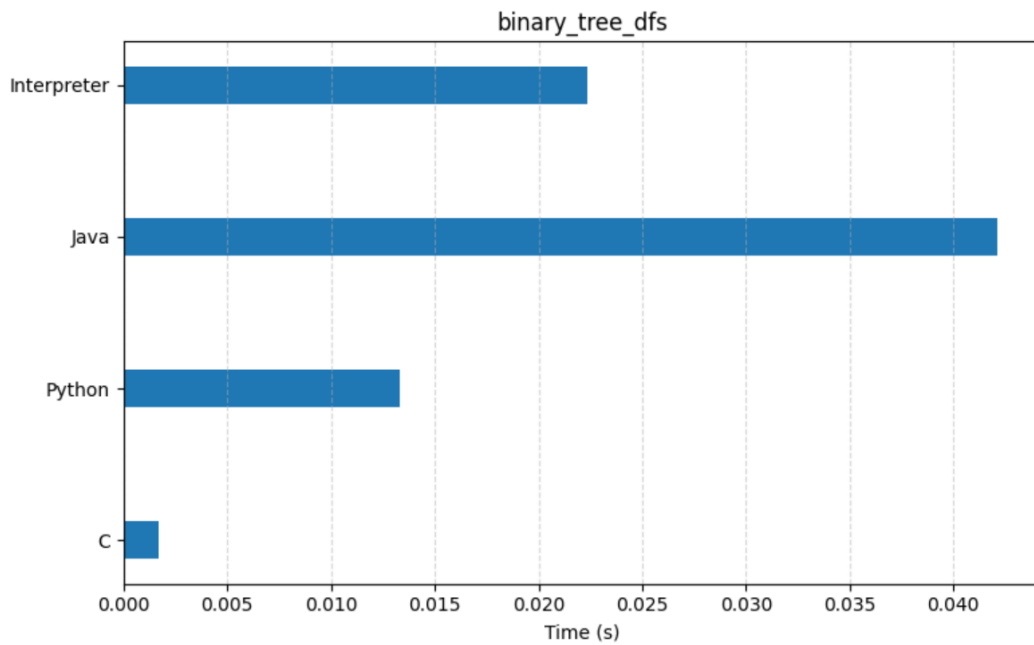
4.4.2 버블 정렬

	C	Python	Java	mU-Code Interpreter
실험 1	1.573	39.235	56.591	7.682
실험 2	2.074	36.644	44.767	7.894
실험 3	2.174	37.415	46.867	7.938
평균	1.940	37.765	49.408	7.838



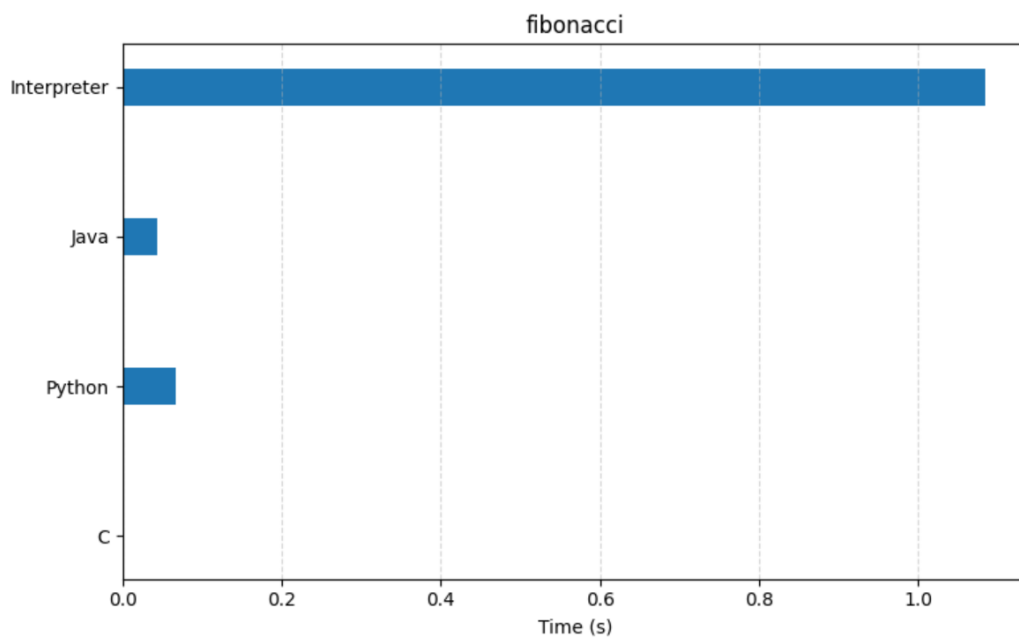
4.4.3 이진 트리 DFS

	C	Python	Java	mU-Code Interpreter
실험 1	1.712	13.017	32.110	22.777
실험 2	1.578	13.413	42.060	22.304
실험 3	1.862	13.444	52.267	22.045
평균	1.717	13.291	42.146	22.375



4.4.4 피보나치 수열

	C	Python	Java	mU-Code Interpreter
실험 1	2.526	67.384	45.289	1094.479
실험 2	2.202	66.179	42.200	1061.605
실험 3	2.148	66.868	44.024	1098.952
평균	2.292	66.810	43.838	1085.012



4.4.5 메모리 사용량 비교

	C	Python	Java	mU-Code Interpreter
--	---	--------	------	---------------------

버블 정렬	1.1MB	10.1 MB	38.9 MB	10.7 MB
이진 트리 DFS	1.1 MB	10.6 MB	39.0 MB	10.7 MB
피보나치 수열	1.1 MB	10.1 MB	39.2 MB	10.8 MB

4.4.6 분석

mU-Code Interpreter와 흔히 사용하는 C,Python,Java의 실행 시간, 메모리 사용량을 비교해 보았다.

버블 정렬 실험에서는 C가 평균 1.94ms로 가장 빠른 성능을 보였고 mU-Code Interpreter는 7.84ms 수준으로 Python, Java 보다 빠르게 수행되었다. 이는 버블 정렬이 단순한 반복문 알고리즘이기 때문에 간단한 스택 기반 구조인 mU-Code Interpreter에서 효율적으로 처리되었기 때문이다.

이진 트리 DFS에서는 mU-Code Interpreter의 평균 실행 시간이 22.3ms로 Python 대비 1.6배 정도 느렸다. DFS는 재귀 호출이 있어, 함수 호출과 스택 프레임에 대한 비용이 높은 mU-Code Interpreter에서 오버헤드가 나타난 것으로 보인다.

피보나치 수열 실험에서는 mU-Code Interpreter의 평균 실행 시간은 1085ms로 타 언어에 비해 훨씬 많은 시간이 소요되었다. 이는 mU-Code Interpreter의 함수 호출 오버헤드가 커, 수많은 재귀 호출이 반복되는 피보나치 수열에서 이런 단점이 크게 작용했기 때문이다.

메모리에 대해서는 10.7MB 정도로 Python과 거의 비슷한 수준으로 실행 프로그램에 따라 거의 달라지지 않는 모습을 보였다. 이는 CPU 스택, 메모리 등 고정적인 메모리 할당을 사용하고 실행 프로그램에 따라 달라지는 부분이 없기 때문인 것으로 보인다.

5. 결 론

5.1 핵심 성과 요약

이번 프로젝트에서는 mU-Code로 작성된 프로그램을 실행할 수 있는 가상 머신 기반 인터프리터를 설계 및 구현했다. mU-Code 프로그램을 실행하기 위한 기능들을 어셈블, 실행, 파일 입출력 등의 모듈로 나눠 추후 유지보수나 확장이 편하게 했다. 또한 GUI를 구현해 사용자가 쉽게 프로그램을 실행하고 그 과정을 살펴볼 수 있다. 입력받은 mU-Code에 문제가 있어 어셈블 또는 실행 중 오류가 발생했을 때는 오류를 감지해 처리하고 안전하게 종료될 수 있도록 했다.

5.2 의의 및 활용 가능성

mU-Code Interpreter는 다양한 mU-Code 프로그램을 실제로 실행할 수 있다는 점에서 의의가 있다. mU-Code 프로그램을 테스트해 보고 싶은 사용자나 스택 기반의 가상 머신 구조를 학습하고 싶은 사용자는 이 인터프리터를 사용해 mU-Code 프로그램을 실행하고, 명령어 처리, 스택의 변화 과정 등을 직접 실행하며 살펴볼 수 있다. 이를 통해서 프로그래밍 언어 개발이나 학습을 위한 도구로 사용할 수 있다.

5.3 한계점 및 향후 개선 과제

현재 프로그램은 사용자 학습에 초점이 맞춰져 있어 오류가 발생하면 메시지 출력만 해주는 기본적인 디버그 기능만 있다는 한계가 존재한다. 그래서 다른 컴파일러처럼 breakpoint, 오타 수정 제안 등 다양한 디버그 기능을 추가하는 식으로 개발자를 위한 기능을 추가해 확장하는 것이 좋을 것이다. 또한, GUI에서 메모리 배열을 보여주지만 각각 데이터가 독립적으로 되어 있어 학습자가 한눈에 이해하기 어려운 건 사실이다. 그래서 memory 영역의 데이터를 라벨, 변수, 레지스터 등을 연관 지어서 표시하는 것이 학습 효율을 높일 것 같다.

프로그램에서 보인 문제점은 GUI로 실행 시 변경 이력이 많아지면 메모리를 과다하게 사용한다는 점이다. 이는 이미 memory 배열을 아주 크게 할당하기 때문에 생긴 문제이다. 이를 위해 변경 기록을 파일로 저장하는 식으로 개선할 수 있다. 또한, 변경 이력을 기록하는 과정에서 실행 시간이 지연되는 문제가 발생하는데 이를 멀티 스레드로 구현하고 변화 플래그로 추가하는 식으로 개선할 수 있을 것이다.

5.4 기대 효과

mU-Code Interpreter는 학습자와 개발자에게 새로운 도구를 제공해 효율을 높일 수 있을 것으로 기대된다. 먼저, GUI를 통해 각 실행 단계별 메모리, 스택의 변화 레지스터의 변화 등을 직관적으로 확인할 수 있어 프로그래밍 언어나 가상 머신 구조에 대한 이해도를 높일 수 있다. 또한 mU-Code를 실행하고 디버깅할 수 있는 환경을 제공함으로써 이 mU-Code를 사용하는 프로그래밍 언어 개발하거나 mU-Code로 프로그램 작성하고자 하는 사용자에게 도움을 줄 수 있다. 향후 mU-Code Interpreter에 명령어 추가, 디버깅 기능 확장, 최적화 등으로 발전시켜 좀 더 실전성 있게 사용할 수 있는 인터프리터의 기반이 될 수 있을 것이다.

6. 자체 평가서

6.1 자체 분석 및 평가

이강석의 자체 분석 및 평가

mU-Code Interpreter 프로젝트를 진행하면서 문제 분석이나 설계 단계에서 충분한 토의를 진행되어 구현 단계가 큰 지체 없이 진행된 것 같다. 막히는 부분이나 설계를 변경해야 하는 문제가 생기더라도 의사소통이 원활하게 진행되고 다양한 아이디어가 나와 전체적으로 무난하게 진행된 것 같다. 팀원이 단순히 적당히 기능만 구현하고 끝내는 것이 아닌 완성도를 높이기 위해서 보완할 부분을 적극적으로 제안해주어서 나 또한 최대한 완성도를 높이려고 할 수 있었다. 예전에 Github 사용법을 배운 적은 있지만 실제로 협업에 사용한 적은 없었다. 그래서 이번 프로젝트에서는 Github를 적극적으로 활용해 브랜치 분리, Pull Request, Issue 등 협업 도구를 이용해 작업하고 의사소통을 나누는 등 협업 과정을 자세히 경험해 볼 수 있었다. 팀원이 Github에 대해서 잘 알고 있어 리포지토리 관리나 브랜치 전략 등 협업에 필요한 다양한 내용들을 배울 수 있었다. 또한 코드 부분에서는 각 함수들의 기능을 테스트할 수 있는 테스트 코드나 전역 변수나 구조체를 한 곳에 묶어 관리하는 Context 구조체 등 다양한 기술들도 배울 수 있었다.

실행 부분에서 명령어를 실제로 처리할 함수를 호출할 때, 처음으로 함수 포인터를 사용해 보았다. 처음에는 switch case 문으로 구현할까 생각했지만 명령어의 종류가 약 30개 정도 되다 보니 성능이 나빠질 것 같았다. 마침 리눅스 수업에서 함수 포인터를 사용하는 것을 처음 보아서 함수 포인터를 사용하면 쉽게 성능을 높일 수 있을 것 같았다. 따라서 명령어 처리 함수 호출에 함수 포인터를 사용했고 안정적으로 성능도 높일 수 있었다

권기영의 자체 분석 및 평가

Github를 통한 협업이 자연스럽게 이루어져 만족스러웠다. 이번에는 설계와 역할 분담이 명확했고, 그 덕에 서로의 작업 범위를 이해한 상태에서 개발을 진행할 수 있었다. 예전에 Github로 프로젝트를 했을 때는 기능 구분 없이 코드를 섞어 쓰다 보니 충돌과 혼란이 많았는데, 그 문제의 원인이 플랫폼이 아니라 요구사항을 제대로 분석하지 않은 상태에서 개발을 시작한 것임을 이번 프로젝트를 통해 깨달았다.

처음엔 ‘왜 요구사항 분석을 굳이 해야 하지?’라는 생각이 있었지만, 막상 해보니 어떤 기능을 개발하든 가장 먼저 요구사항을 생각하는 습관이 생겼다. 이 과정 덕분에 다른 과제에서도 오해 없이 개발을 시작할 수 있었고 구현 범위도 명확해 속도도 빨라졌다. 역할 분담도 초반부터 확실하게 끝내서 불필요한 조율 없이 각자 개발에 집중할 수 있었고, 나는 맡은 기능을 더 고도화하는 데 시간을 쓸 수 있었다. 또한, 팀장도 비슷한 목표 의식을 가지고 있어 의견 조율이 수월했다.

한편, 평소 동적 할당을 선호하지 않았지만, 이번에는 전체 줄 관리를 위해 사용했다. 하지만 메모리 해제 관리가 미흡해 코드가 꼬이고 중복 해제 오류가 발생했다. 결국 AI를 활용해 문제를 해결했으며, 앞으로는 동적 할당 시 주석을 통한 명확한 관리 습관을 들여야 한다는 점을 깨달았다. 또한, 보고서를 쓰면서 코드를 다시 차분히 읽어보는 기회가 있었는데 기능을 추가하는 데에 너무 혈안이 되었던지 작동은 하나 코드 논리가 미흡한 부분이 보였다. 코드 리뷰 때마다 귀찮아서 살짝 대중 본 점이 아쉬웠으며 반성해야겠다고 생각했다.

새롭게 습득한 기술은 GUI 반응성을 유지하기 위해 실행 엔진을 독립 스레드로 분리하고, g_async_queue로 스레드 간 I/O를 동기화한 구조를 설계한 것이다. UI 스레드와 실행 스레드의 경계를 명확히 하기 위해, 모든 UI 갱신은 g_idle_add를 통해 메인 스레드에서만 처리되도록 했다. 이렇게 분리한 이유는 어셈블과 실행은 시간이 오래 걸릴 수 있어 메인 스레드에서 처리하면 GUI가 멈추기 때문이다. 입출력은 메시지 큐 기반으로 비동기 처리해, UI 스레드는 입력을 queue에 push하고 실행 스레드는 필요할 때 pop으로 입력이 올 때까지 block 하면서 기다리도록 구성했다. 이 방식으로 양쪽이 서로의 흐름을 방해하지 않고 독립

적으로 동작함을 보장했다.

6.2 팀원 역할 및 기여도

이강석

- 역할: CLI 구현, lst 파일 처리, Runner, 문제 개요, 전체 프로그램 구조, 벤치마크 기능
- 공통 수행: 설계/최종 보고서 및 PPT, 코드 피드백, 테스트 코드
- 공통 기여도: 50%

권기영

- 역할: GUI 구현, uco 파일 처리, Assembler, 요구 사항 정의, 리눅스 패키지 배포
- 공통 수행: 설계/최종 보고서 및 PPT, 코드 피드백, 테스트 코드
- 공통 기여도: 50%

7. 부 록

GitHub Repository: [jhan117/mU-Code-Interpreter](https://github.com/jhan117/mU-Code-Interpreter)

Release: [Release mU-Code-Interpreter v1.0.3](#)

회의록: [mU-Code-Interpreter/docs/minutes.md](#)

8. 참고문헌

윤OO, “소프트웨어 프로젝트 강의 자료: 3단계 프로젝트,” 국립금오공과대학교 컴퓨터공학과, 2025.

안OO, “컴퓨터구조 강의 자료: 2장 CPU의 구조와 기능,” 국립금오공과대학교 컴퓨터공학과, 2025.

TCP School, “C 언어 스택 프레임과 메모리 구조,” http://www.tcpschool.com/c/c_memory_stackframe.

최OO, “리눅스 활용 및 프로그래밍 강의 자료: 3장 파일 다루기,” 국립금오공과대학교 컴퓨터공학과, 2025.

JRL, "Trim a string in C," <https://stackoverflow.com/questions/656542/trim-a-string-in-c>.

기만결, “strcmp(),strcpy(),strncpy(),strdup() 함수,” https://blog.naver.com/khg27_2000/50090047187.

Gtk-3.0, “Gtk - 3.0: Widget Gallery”, https://docs.gtk.org/gtk3/visual_index.html.

GLib, “GAsyncQueue,” <https://docs.gtk.org/glib/struct.AsyncQueue.html>.

GLib, “Threads,” <https://docs.gtk.org/glib/threads.html>.

GLib, “GLib.Thread,” <https://docs.gtk.org/glib/struct.Thread.html>.

AI 활용 내역

AI 종류	사용 단계	질문 내용	응답 내용
Copilot	설계	어셈블 도중 발생할만한 오류 목록 알려줘	오류 이름 및 내용 목록
Copilot	구현 (주로 Gtk/GLib 문서에 활용)	공식 문서 내용 요약/번역 해줘	공식 문서 내용 요약 및 번역
ChatGPT-5	구현	lst 저장 시에 계속 중복 해제 뜨는 데 어떻게 해야 하지	가변 인자를 활용해 임시 버퍼로 문자열 처리하는 함수 내용 (lst_file.c 의 <code>static int appendFormat(char **buf, int *len, int *cap, const char *fmt, ...)</code> 함수 내용)
ChatGPT-5	구현	Test/GUI 전용 Makefile 작성해줘	Makefile 내용
ChatGPT-5	구현	C 배열에서 특정 값만 따로 초기화 하는 방법	<code>arr[] = { [20] = 1, [25] = 2 }</code> 등의 초기화 방법
ChatGPT-5	구현	~에 쓸 변수/함수/상수명 추천해줘	ERR_XXX_XXX 등의 이름
ChatGPT-5	구현	<code>g_async_queue_push: assertion 'data' failed</code>	GINT_TO_POINTER(0)은 NULL이므로 push 불가,

		0을 입력하면 오류가 뜨는 이유	GAsyncQueue는 포인터만 저장 따라서 정수를 저장 하려면 포인터 포장이 필 요함
ChatGPT-5	구현	커밋 메시지 추천(영어, 양 식)	main에 존재하는 커밋 로 그 대부분에 사용함